生物实验室管理平台设计计划书

1. 项目概述

1.1 项目背景

本系统面向学校生物实验室管理场景，服务对象包括生物教师与实验管理员。系统旨在实现仪器台账数字化管理、实验预约流程规范化、实验过程记录电子化，以及仪器借用、维修、损耗、采购等日常管理的信息化，提升实验室管理效率。

1.2 建设目标

- 建立完整的生物仪器电子台账，实现仪器全生命周期管理。
- 为生物教师提供实验预约、使用登记、分组实验记录及实验报告单生成的一站式服务。
- 为实验管理员提供仪器借用、维修、损耗、采购申报及实验通知单发送等管理功能。
- 支持学校局域网内通过浏览器访问，同时支持微信小程序端使用。

1.3 适用范围

本系统适用于学校生物实验室的日常教学与管理，不涉及校外互联网公开访问场景。

---

2. 用户角色与权限设计

2.1 生物教师

- 登录系统，查看个人信息。
- 查询仪器台账及可用状态。
- 提交分组实验预约申请。
- 登记实验使用情况，记录分组实验过程。
- 生成并查看实验报告单。
- 接收实验通知单及系统公告。
- 申请仪器借用。

2.2 实验管理员

- 管理仪器台账（新增、编辑、删除、查询）。
- 审核教师提交的实验预约申请。
- 登记仪器借用、归还记录。

- 记录仪器维修、损耗、报废情况。
- 提交仪器采购申报。
- 发送实验通知单。
- 管理教师及管理员账号。
- 查看统计报表。

2.3 系统管理员（可选）

- 管理系统基础配置。
- 管理用户账号与角色权限。
- 查看系统操作日志。

---

3. 功能模块设计

3.1 用户管理模块

- 用户注册与登录。
- 角色分配（教师、管理员）。
- 密码修改与找回。
- 个人信息维护。

3.2 仪器台账管理模块

- 仪器信息录入：编号、名称、型号、分类、数量、存放位置、状态、购置日期等。
- 仪器查询与筛选。
- 仪器状态更新：正常、借出、维修中、损耗、报废。
- 仪器批量导入/导出（可选）。

3.3 实验预约管理模块

- 教师提交实验预约：选择实验室、填写实验名称、时间、学生人数、备注等。
- 管理员审核预约：通过、驳回、调整时间。
- 预约状态跟踪：待审核、已通过、已驳回、已完成、已取消。
- 预约日历视图展示。

3.4 实验登记与报告模块

- 实验使用登记：记录实际使用时间、参与班级、分组情况。
- 实验过程记录：填写实验内容、实验结果、异常情况。
- 实验报告单生成：支持在线填写或上传附件，生成标准化报告单。
- 历史报告查询与导出。

3.5 仪器借用管理模块

- 借用申请：教师选择仪器，填写借用时间、用途。
- 借用登记：管理员确认借出，记录借出时间。
- 归还登记：记录归还时间、仪器状态。
- 逾期未还提醒。

3.6 仪器维修管理模块

- 故障上报：记录仪器故障描述、上报时间。
- 维修记录：填写维修时间、维修结果、维修费用。
- 维修状态跟踪：报修中、维修中、已完成、报废。

3.7 仪器损耗管理模块

- 损耗登记：记录损耗仪器、损耗时间、损耗原因、责任人。
- 损耗统计与查询。

3.8 采购申报管理模块

- 采购申请：填写仪器名称、型号、数量、预算、申请理由。
- 采购状态跟踪：草稿、已提交、已审批、已驳回、已采购。
- 采购记录查询。

3.9 实验通知单模块

- 管理员编辑并发送实验通知单。
- 通知单包含：实验名称、时间、地点、所需仪器、注意事项等。
- 教师端接收通知，支持查看历史通知。

3.10 统计报表模块（可选）

- 仪器使用率统计。
- 实验预约频次统计。

• 仪器维修/损耗统计。

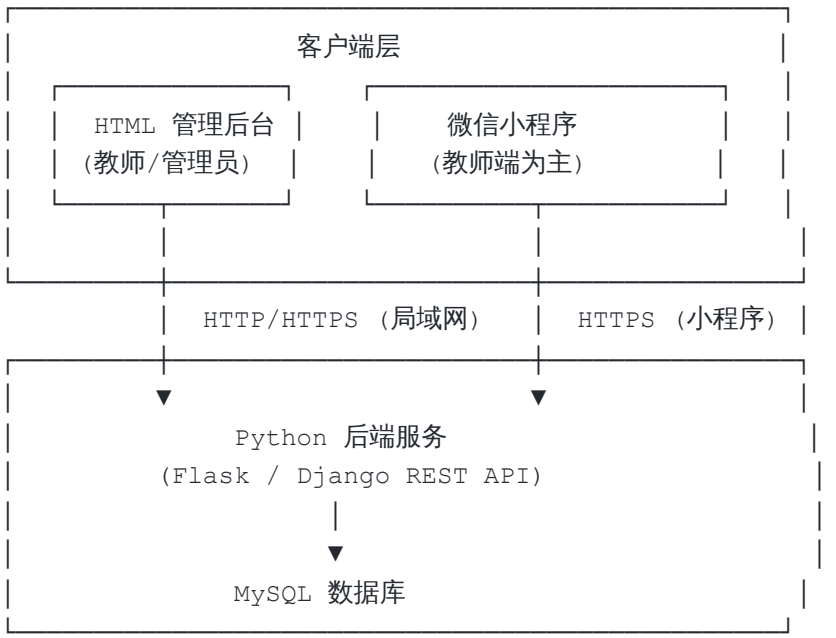
• 采购需求汇总。

---

4. 技术架构设计

4.1 整体架构

Plain Text



4.2 技术选型

层级	技术	说明
后端	Python + Django 或 Flask	Django 推荐，自带后台和权限管理；Flask 更轻量灵活
数据库	MySQL 5.7+	关系型数据库，存储所有业务数据
前端（PC）	HTML + CSS + JavaScript	可使用 Bootstrap 或 Vue.js 构建管理后台
移动端	微信小程序	原生小程序开发，调用后端 API

服务器	Windows / Linux	校内服务器，部署 Python 服务和 MySQL
网络	学校局域网	PC 端通过局域网 IP 访问；小程序需配置合法域名

4.3 接口设计规范

- 采用 RESTful 风格 API。
- 数据格式统一使用 JSON。
- 接口鉴权采用 Token 机制（如 JWT）。
- 示例接口：
  - `GET /api/instruments` — 查询仪器列表
  - `POST /api/reservations` — 提交实验预约
  - `POST /api/login` — 用户登录
  - `GET /api/notices` — 获取通知列表

---

5. 数据库设计

5.1 核心数据表

表名	说明	主要字段
users	用户表	id, username, password_hash, role, name, phone
instruments	仪器台账表	id, code, name, model, category, quantity, location, status
reservations	实验预约表	id, teacher_id, lab_name, experiment_name, start_time, end_time, status
borrow_records	借用记录表	id, instrument_id, teacher_id, borrow_time, return_tim

		e, status
repair_records	维修记录表	id, instrument_id, fault_desc, repair_time, cost, status
loss_records	损耗记录表	id, instrument_id, loss_time, reason, handler_id
purchase_requests	采购申报表	id, instrument_name, model, quantity, budget, status, applicant_id
experiment_reports	实验报告表	id, reservation_id, teacher_id, experiment_name, content, result
notices	通知单表	id, title, content, sender_id, created_at

5.2 表关系说明

- `users` 与 `reservations` ：一对多，一个教师可提交多条预约。
- `users` 与 `borrow_records` ：一对多，一个教师可多次借用仪器。
- `instruments` 与 `borrow_records` ：一对多，一台仪器可被多次借用。
- `instruments` 与 `repair_records` ：一对多。
- `instruments` 与 `loss_records` ：一对多。
- `users` 与 `purchase_requests` ：一对多。
- `reservations` 与 `experiment_reports` ：一对多或一对一。
- `users` 与 `notices` ：一对多，管理员可发送多条通知。

---

6. 部署方案

6.1 服务器环境

- 操作系统：Windows Server 或 Linux（Ubuntu/CentOS）。
- Python 版本：3.9+。
- 数据库：MySQL 5.7+ 或 MariaDB。

- Web 服务器：Nginx（可选，用于反向代理和静态文件服务）。
- 应用服务器：Gunicorn（Flask）或 uWSGI（Django）。

6.2 网络配置

- **PC 端访问**：教师和管理员通过浏览器访问 `http://服务器内网IP:端口` ，例如 `http://192.168.1.100:8000` 。
- **微信小程序访问**：小程序需调用后端 API，要求后端域名经过 ICP 备案并配置 HTTPS。若仅校内使用，可考虑以下替代方案：
  - 使用学校已有的域名配置 HTTPS 反向代理。
  - 使用企业微信或学校公众号 H5 页面替代小程序。
  - 开发阶段使用小程序开发者工具调试，不发布正式版。

6.3 部署步骤

1. 在服务器上安装 Python、MySQL、Git 等基础环境。
2. 创建数据库并导入表结构。
3. 部署 Python 后端服务，配置数据库连接。
4. 启动后端服务，监听 `0.0.0.0` 指定端口。
5. 配置防火墙，放行对应端口。
6. 部署前端 HTML 页面，或使用 Django/Flask 模板渲染。
7. 开发并上传微信小程序，配置服务器域名。
8. 创建初始管理员账号，进行系统测试。

---

7. 安全设计

7.1 用户认证

- 密码采用哈希加密存储（如 bcrypt）。
- 登录接口返回 Token，后续请求携带 Token 鉴权。
- 支持密码修改功能。

7.2 权限控制

- 基于角色的访问控制（RBAC）：教师和管理员拥有不同接口权限。
- 后端对每个接口进行角色校验，防止越权操作。

7.3 数据安全

- 数据库定期备份。
- 敏感操作记录日志（如删除仪器、审核预约）。
- 输入数据校验，防止 SQL 注入和 XSS 攻击。

---

8. 开发计划

第一阶段：需求确认与基础搭建（1–2 周）

- 确认详细需求，完善功能清单。
- 搭建开发环境，创建项目框架。
- 设计数据库表结构并完成建表。
- 实现用户登录与角色权限模块。

第二阶段：核心功能开发（3–5 周）

- 开发仪器台账管理模块。
- 开发实验预约与审核模块。
- 开发仪器借用管理模块。
- 开发 HTML 管理后台基础页面。

第三阶段：扩展功能开发（2–3 周）

- 开发仪器维修、损耗、采购申报模块。
- 开发实验登记与报告单生成模块。
- 开发实验通知单模块。
- 开发统计报表模块（可选）。

第四阶段：小程序开发（2–3 周）

- 设计小程序 UI 界面。
- 开发小程序各功能页面。
- 联调后端 API。
- 小程序测试与优化。

第五阶段：测试与部署（1–2 周）

- 功能测试、性能测试、安全测试。
- 修复 Bug，优化用户体验。
- 部署到校内服务器。
- 用户培训与上线运行。

---

9. 后续扩展建议

- 增加二维码扫描功能，仪器贴码，扫码即可查看信息或登记借用。
- 增加消息推送功能，预约审核结果通过微信模板消息通知教师。
- 增加数据导出功能，支持 Excel 导出仪器台账、报表等。
- 增加多实验室管理，支持多个生物实验室独立预约。
- 增加耗材管理模块，管理试剂、玻璃器皿等消耗品。
- 对接学校统一身份认证系统（如已有）。

---

10. 总结

本系统以 Python 为后端核心，MySQL 为数据存储，HTML 为 PC 端界面，微信小程序为移动端入口，覆盖生物实验室从仪器管理到实验教学的全流程。系统部署于学校局域网内，兼顾安全性与便捷性，能够有效提升生物实验室的管理效率和教学服务水平。